

# Feature Engineering

A Complete Reference Guide for ML Practitioners

Amol Jagtap | amoljagtap3001@gmail.com

| Field         | Detail                                                                |
|---------------|-----------------------------------------------------------------------|
| Day           | Day 23 — Machine Learning                                             |
| Topic         | What is Feature Engineering?                                          |
| Pre-requisite | Bivariate Analysis   EDA   Pandas Profiling                           |
| Next Topic    | Feature Scaling & Standardization                                     |
| Libraries     | pandas, numpy, scipy, sklearn, seaborn, matplotlib                    |
| Dataset Used  | Various toy datasets, custom examples                                 |
| Key Takeaway  | Better features = better model accuracy (even with simple algorithms) |

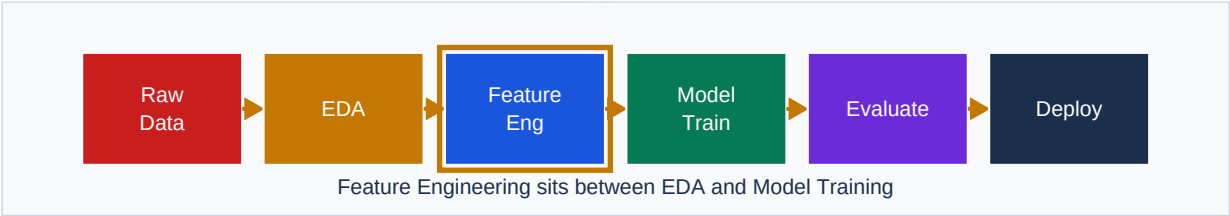


Figure 1 — Feature Engineering sits between EDA and Model Training (highlighted)

# Table of Contents

---

1. What is Feature Engineering?
2. Why Feature Engineering Matters
3. Overview of All Types
4. Feature Transformation
5. Feature Construction
6. Feature Selection
7. Feature Extraction — PCA and LDA
8. Handling Outliers
9. Handling Imbalanced Datasets
10. End-to-End Feature Engineering Workflow
11. Quick Reference Cheat Sheet

Amol Jagtap | amoljagtap3001@gmail.com

# 1. What is Feature Engineering?

---

Feature Engineering is the process of using domain knowledge and data analysis to create, transform, select, or extract input variables (features) from raw data so that machine learning models can learn from them more effectively. It is the bridge between raw data and a well-performing ML model.

| Term               | Simple Definition                                  | Example                                                        |
|--------------------|----------------------------------------------------|----------------------------------------------------------------|
| Feature            | An input variable fed to the model                 | 'Age', 'Income', 'City'                                        |
| Label / Target     | The output the model tries to predict              | 'Survived', 'Price', 'Churn'                                   |
| Raw Data           | Data as collected — messy, incomplete, wrong types | CSV with NaN, text dates, mixed categories                     |
| Engineered Feature | A new or improved variable ready for modelling     | 'Age_Group', $\log(\text{Price})$ , 'Days_Since_Last_Purchase' |

■ *The quality of features you feed the model matters MORE than the choice of algorithm. A simple Logistic Regression with great features beats a complex Neural Network with bad features.*

## 2. Why Feature Engineering Matters

---

Raw data collected from the real world is rarely in the right shape for ML. Consider a house price dataset: the raw data has 'date\_built' (a string), 'area\_sqft' (numbers ranging from 200 to 50,000), and 'city' (text categories). None of these are ready for a model. Feature Engineering fixes all of this.

| Problem in Raw Data          | Feature Engineering Solution | Benefit                           |
|------------------------------|------------------------------|-----------------------------------|
| Skewed numeric distributions | Log / sqrt transform         | Many algorithms assume normality  |
| Date stored as a string      | Extract year, month, weekday | Captures seasonal patterns        |
| Categorical text columns     | One-hot or ordinal encoding  | Algorithms need numbers           |
| 100+ features (curse of dim) | Select top 20 by importance  | Faster training, less overfitting |
| Outliers pulling predictions | Cap or remove outliers       | Robust, stable model              |
| Class imbalance (99% vs 1%)  | SMOTE / oversampling         | Model learns minority class       |
| Missing values               | Imputation strategies        | No crashes, unbiased estimates    |

### 3. Overview of All Types

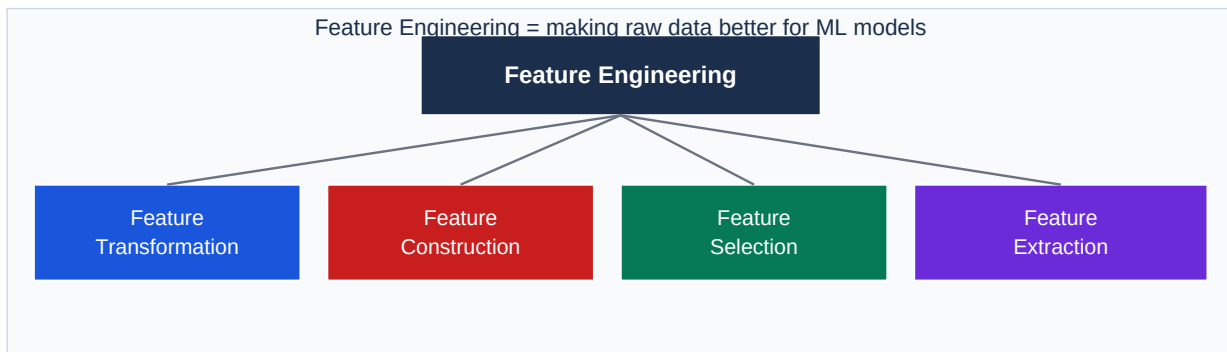


Figure 2 — Four main branches of Feature Engineering

| Type                   | What it does                                    | When to use                                             |
|------------------------|-------------------------------------------------|---------------------------------------------------------|
| Feature Transformation | Changes the distribution or scale of a feature  | When data is skewed, has outliers, or needs rescaling   |
| Feature Construction   | Creates brand-new features from existing ones   | When raw columns don't directly capture useful patterns |
| Feature Selection      | Removes irrelevant or redundant features        | When you have too many features (high dimensionality)   |
| Feature Extraction     | Reduces many features into fewer key components | When features are correlated (PCA, LDA, autoencoders)   |

## 4. Feature Transformation

Feature Transformation changes how the values of a feature are distributed without adding or removing features. The most common reason is to fix skewness — many ML algorithms perform better when numeric features follow a normal (bell curve) distribution.

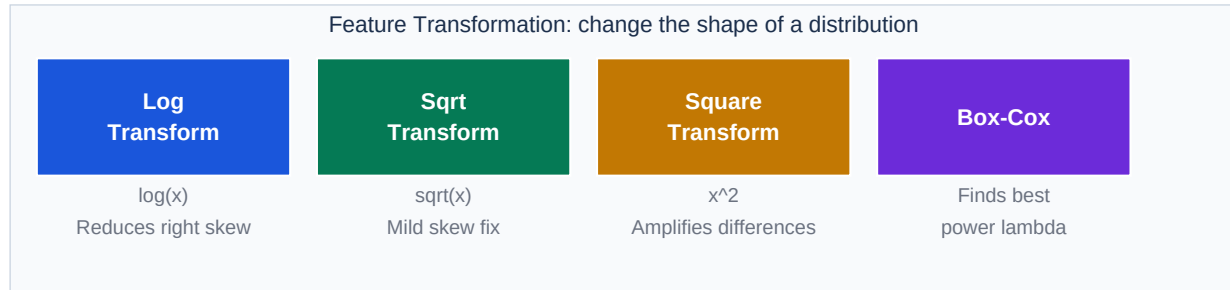


Figure 3 — Four common transformation types and when to use each

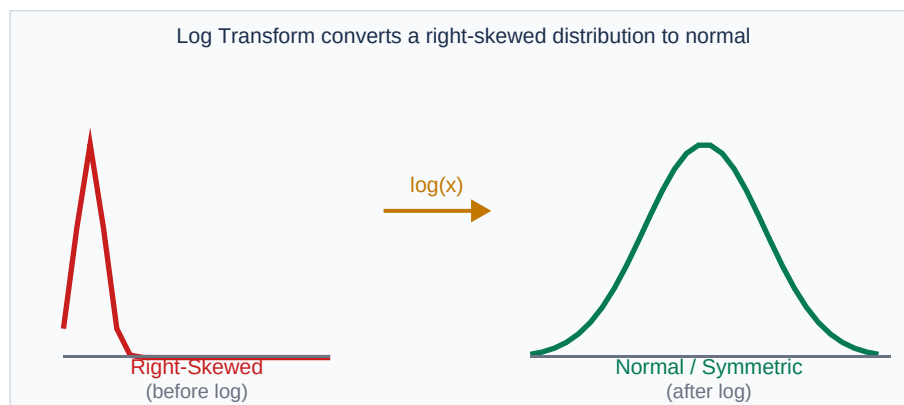


Figure 4 — Log transform converts right-skewed data to a symmetric distribution

### 4.1 Log Transformation

The most popular transformation. Apply when data has a strong right skew (long tail on the right). Common in price, income, and population data.

**$y_{\text{new}} = \log(x)$  or  $\log(x + 1)$  if  $x$  can be 0**

```
import numpy as np import pandas as pd # Apply log transform (add 1 to avoid log(0))
df['log_price'] = np.log1p(df['price']) # log(1 + price) # Compare skewness before and
after print('Before:', df['price'].skew().round(3)) print('After :',
df['log_price'].skew().round(3)) # Visualise import matplotlib.pyplot as plt fig, axes
= plt.subplots(1, 2, figsize=(10, 4)) df['price'].hist(ax=axes[0], bins=30,
color='salmon') axes[0].set_title('Before: Skewed') df['log_price'].hist(ax=axes[1],
bins=30, color='steelblue') axes[1].set_title('After: Log Transform') plt.show()
```

### 4.2 Square Root and Power Transforms

```
# Square root (milder than log) df['sqrt_area'] = np.sqrt(df['area']) # Square (for
right-skewed with small values) df['age_sq'] = df['age'] ** 2 # Cube root (handles
negative values) df['cbrt_col'] = np.cbrt(df['col'])
```

### 4.3 Box-Cox and Yeo-Johnson Transforms

Box-Cox automatically finds the best power transformation (lambda) to make data as normal as possible. Yeo-Johnson is an extension that also handles zero and negative values.

```
from sklearn.preprocessing import PowerTransformer # Box-Cox (only for positive values)
pt_bc = PowerTransformer(method='box-cox') df['bc_col'] =
pt_bc.fit_transform(df[['col']]) # Yeo-Johnson (works with zeros and negatives) pt_yj =
PowerTransformer(method='yeo-johnson') df['yj_col'] = pt_yj.fit_transform(df[['col']])
```

| Transform   | Formula                     | Works on      | Best For                  |
|-------------|-----------------------------|---------------|---------------------------|
| Log         | $\log(x)$ or $\log_{10}(x)$ | Positive only | Income, price, population |
| Sqrt        | $\sqrt{x}$                  | Positive only | Counts, mild skew         |
| Square      | $x^2$                       | Any           | Polynomial relationships  |
| Cube Root   | $x^{1/3}$                   | Negative too  | Sales, profit data        |
| Box-Cox     | Auto power                  | Positive only | When unsure; automated    |
| Yeo-Johnson | Auto power                  | Any (neg OK)  | General purpose           |

## 5. Feature Construction

Feature Construction means creating brand-new columns from existing ones. These new features capture patterns or relationships that the raw columns alone do not reveal. This is where domain knowledge is most valuable.

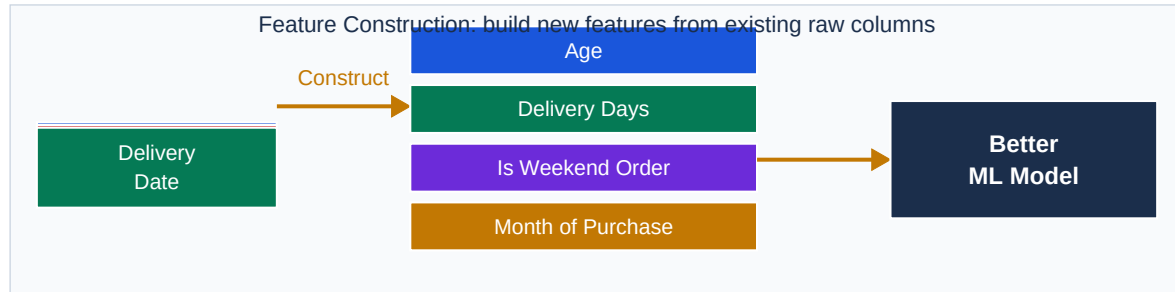


Figure 5 — Raw date columns -> new constructed features

### Common Construction Techniques

| Technique           | Raw Feature(s)      | New Feature                       | Captures                 |
|---------------------|---------------------|-----------------------------------|--------------------------|
| Date decomposition  | Date of Birth       | Age, Birth_Month, Is_Weekend      | Temporal patterns        |
| Interaction         | Height, Weight      | $BMI = Weight/Height^2$           | Combined effect          |
| Ratios              | Sales, Visits       | $Conversion\_Rate = Sales/Visits$ | Efficiency metric        |
| Aggregations        | Transaction history | Total_Spend, Avg_Order_Value      | Customer behaviour       |
| Binning / Bucketing | Age (continuous)    | Age_Group (Young/Mid/Senior)      | Non-linear patterns      |
| Text features       | Product Description | Word Count, Has_Keyword           | NLP signals              |
| Lag features        | Sales (time series) | Sales_Lag_7 (7 days ago)          | Temporal autocorrelation |

```
import pandas as pd
import numpy as np

# 1. Extract from date
df['Order_Date'] = pd.to_datetime(df['Order_Date'])
df['Year'] = df['Order_Date'].dt.year
df['Month'] = df['Order_Date'].dt.month
df['Day_of_Week'] = df['Order_Date'].dt.weekday
# 0=Mon
df['Is_Weekend'] = df['Day_of_Week'].isin([5,6]).astype(int)

# 2. BMI from height and weight
df['BMI'] = df['Weight_kg'] / (df['Height_m'] ** 2)

# 3. Age from date of birth
df['Age'] = (pd.Timestamp.now() - df['DOB']).dt.days // 365

# 4. Binning age into groups
bins = [0, 18, 35, 60, 120]
labels = ['Minor', 'Young', 'Middle', 'Senior']
df['Age_Group'] = pd.cut(df['Age'], bins=bins, labels=labels)

# 5. Interaction feature
df['Revenue_per_Visit'] = df['Revenue'] / df['Visits'].replace(0, 1)

# 6. Days between two dates
df['Delivery_Days'] = (df['Delivery_Date'] - df['Order_Date']).dt.days
```



## 6. Feature Selection

Feature Selection removes features that are irrelevant, redundant, or harmful to model performance. More features is NOT always better — irrelevant features add noise and slow training (the Curse of Dimensionality).

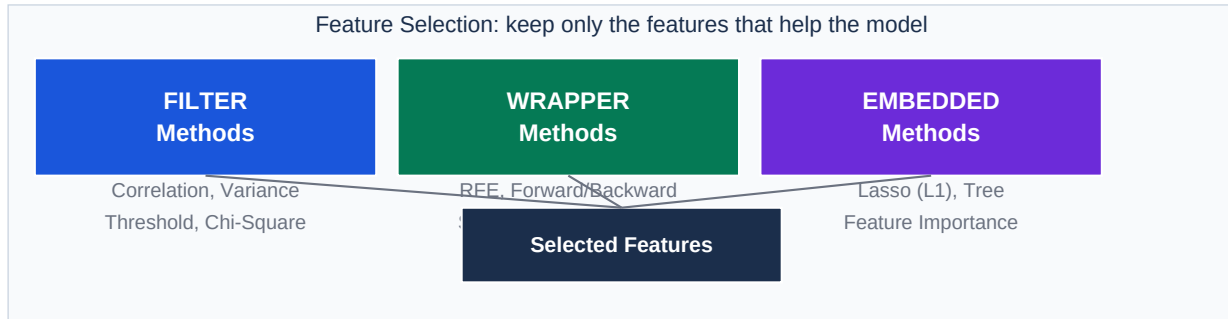


Figure 6 — Three families of feature selection methods

### 6.1 Filter Methods (no model required)

Filter methods rank features using a statistical score and select the top ones. They are fast, model-agnostic, and run before training.

```
import pandas as pd
import numpy as np

# 1. Variance Threshold – remove near-constant features
from sklearn.feature_selection import VarianceThreshold
sel = VarianceThreshold(threshold=0.01)
X_var = sel.fit_transform(X)

# 2. Correlation filter – remove highly correlated pairs
corr = pd.DataFrame(X).corr().abs()
upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
to_drop = [c for c in upper.columns if any(upper[c] > 0.90)]
X_filtered = pd.DataFrame(X).drop(columns=to_drop)

# 3. SelectKBest – pick top-K features by score
from sklearn.feature_selection import SelectKBest, f_classif
sel = SelectKBest(score_func=f_classif, k=10)
X_best = sel.fit_transform(X, y)
print('Selected features:', sel.get_support(indices=True))
```

### 6.2 Wrapper Methods (uses a model)

Wrapper methods train a model repeatedly with different feature subsets to find the best combination. Slower but often more accurate.

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

# RFE – Recursive Feature Elimination model
model = LogisticRegression(max_iter=500)
rfe = RFE(model, n_features_to_select=10)
X_rfe = rfe.fit_transform(X_train, y_train)
print('Selected:', rfe.support_)
print('Ranking:', rfe.ranking_)
```

### 6.3 Embedded Methods (built into the algorithm)

```
from sklearn.ensemble import RandomForestClassifier
import pandas as pd
import matplotlib.pyplot as plt

# Random Forest Feature Importance
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
imp = pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascending=False)
print(imp)
imp.head(15).plot(kind='barh')
plt.title('Feature Importances (Random Forest)')
plt.show()

# Lasso (L1) – sets coefficients of weak features to exactly 0
from sklearn.linear_model import Lasso
lasso = Lasso(alpha=0.01)
lasso.fit(X_train, y_train)
selected = X.columns[lasso.coef_ != 0]
print('Lasso selected features:', selected)
```

```
selected.toList()
```

Amol Jagtap | amoljagtap3001@gmail.com

## 7. Feature Extraction — PCA and LDA

Feature Extraction is different from Feature Selection. Instead of removing original features, it creates a completely new set of fewer features (called components) that capture most of the information. PCA (Principal Component Analysis) is the most popular method.

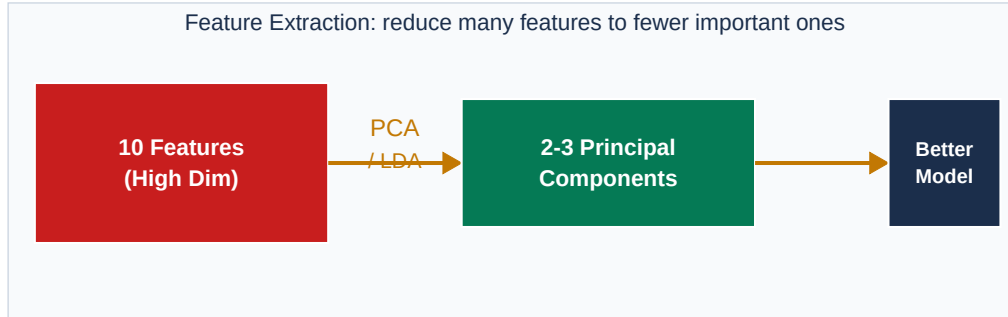


Figure 7 — PCA reduces 10 correlated features to 2-3 principal components

### 7.1 PCA — Principal Component Analysis

PCA finds the directions in the data that have the highest variance (the most information) and projects the data onto those directions. The result is a smaller set of uncorrelated components.

```
from sklearn.decomposition import PCA from sklearn.preprocessing import StandardScaler
# Step 1: Always scale before PCA scaler = StandardScaler() X_scaled = scaler.fit_transform(X_train)
# Step 2: Apply PCA pca = PCA(n_components=2) # keep 2 components X_pca = pca.fit_transform(X_scaled)
# Step 3: Check explained variance print('Explained variance ratio:', pca.explained_variance_ratio_)
print('Total variance explained:', pca.explained_variance_ratio_.sum().round(3))
# Find n_components to explain 95% of variance pca_full = PCA(n_components=0.95) # keeps enough for 95%
X_95 = pca_full.fit_transform(X_scaled) print('Components needed for 95%:', pca_full.n_components_)
```

### 7.2 LDA — Linear Discriminant Analysis

LDA is a supervised method (it uses the labels  $y$ ). It finds directions that best separate the classes. Use LDA for classification problems; PCA for any task.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis lda = LinearDiscriminantAnalysis(n_components=2)
X_lda = lda.fit_transform(X_train, y_train)
print('LDA components shape:', X_lda.shape)
```

|                | PCA                                                 | LDA                       |
|----------------|-----------------------------------------------------|---------------------------|
| Type           | Unsupervised                                        | Supervised (needs labels) |
| Goal           | Maximise variance                                   | Maximise class separation |
| Use for        | Any task                                            | Classification only       |
| Scaling        | Required (StandardScaler)                           | Required                  |
| Max components | $\min(n_{\text{samples}}, n_{\text{features}}) - 1$ | $n_{\text{classes}} - 1$  |

## 8. Handling Outliers

Outliers are data points that lie far from the rest of the data. They can be genuine rare events, data entry errors, or measurement mistakes. Outliers distort mean, standard deviation, and model weights — so they must be detected and handled carefully.



Figure 8 — IQR method vs Z-Score method for detecting outliers

### 8.1 Detecting Outliers

```
import numpy as np import pandas as pd # --- Method 1: IQR (Interquartile Range) --- Q1 = df['col'].quantile(0.25) Q3 = df['col'].quantile(0.75) IQR = Q3 - Q1 lower = Q1 - 1.5 * IQR upper = Q3 + 1.5 * IQR outliers_iqr = df[(df['col'] < lower) | (df['col'] > upper)] print(f'IQR outliers: {len(outliers_iqr)}') # --- Method 2: Z-Score --- from scipy import stats z_scores = np.abs(stats.zscore(df['col'].dropna())) outliers_z = df[z_scores > 3] print(f'Z-Score outliers: {len(outliers_z)}')
```

### 8.2 Handling Outliers — 4 Strategies

| Strategy   | What it does                    | Code example                                            |
|------------|---------------------------------|---------------------------------------------------------|
| Remove     | Delete outlier rows             | <code>df = df[df['col'].between(lower, upper)]</code>   |
| Cap / Clip | Replace with boundary values    | <code>df['col'].clip(lower, upper, inplace=True)</code> |
| Impute     | Replace with median             | <code>df.loc[mask, 'col'] = df['col'].median()</code>   |
| Transform  | Log/sqrt reduces outlier effect | <code>df['col'] = np.log1p(df['col'])</code>            |

```
# Strategy 1: Remove outliers df_clean = df[(df['col'] >= lower) & (df['col'] <= upper)] # Strategy 2: Cap/clip (Winsorization) df['col'] = df['col'].clip(lower=lower, upper=upper) # Strategy 3: Replace outliers with median median = df['col'].median() df.loc[(df['col'] < lower) | (df['col'] > upper), 'col'] = median # Strategy 4: Visualise outliers with a box plot import seaborn as sns sns.boxplot(data=df, y='col') import matplotlib.pyplot as plt plt.title('Outlier Detection via Box Plot') plt.show()
```

■ Do not blindly remove all outliers. In fraud detection, medical data, or rare event prediction, outliers ARE the signal you want to detect.

## 9. Handling Imbalanced Datasets

A dataset is imbalanced when one class (e.g. fraud=1) has far fewer samples than another (e.g. normal=0). A naive model can get 99% accuracy by simply predicting the majority class every time — but it would miss every fraud case. This is a critical real-world problem.

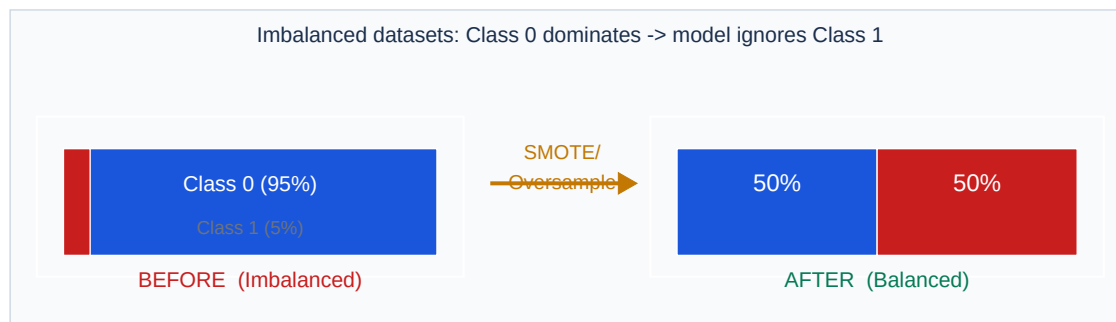


Figure 9 — Before (95/5 split) vs After (50/50) balancing with SMOTE

### 9.1 Detecting Class Imbalance

```
# Check class distribution print(df['target'].value_counts())
print(df['target'].value_counts(normalize=True).round(3))
```

### 9.2 Strategy 1 — Oversampling the Minority Class

SMOTE (Synthetic Minority Oversampling TEchnique) creates synthetic new samples for the minority class by interpolating between existing samples.

```
# pip install imbalanced-learn from imblearn.over_sampling import SMOTE sm =
SMOTE(random_state=42) X_res, y_res = sm.fit_resample(X_train, y_train) print('Before
SMOTE:', pd.Series(y_train).value_counts().to_dict()) print('After SMOTE:',
pd.Series(y_res).value_counts().to_dict())
```

### 9.3 Strategy 2 — Undersampling the Majority Class

```
from imblearn.under_sampling import RandomUnderSampler rus =
RandomUnderSampler(random_state=42) X_res, y_res = rus.fit_resample(X_train, y_train)
```

### 9.4 Strategy 3 — Class Weight Parameter

Many sklearn models accept a `class_weight` parameter. Setting it to 'balanced' automatically penalises mistakes on the minority class more.

```
from sklearn.linear_model import LogisticRegression from sklearn.ensemble import
RandomForestClassifier # LogisticRegression with balanced weights model =
LogisticRegression(class_weight='balanced') # RandomForest with balanced weights model
= RandomForestClassifier(class_weight='balanced', n_estimators=100)
```

| Strategy           | How it works                       | Pros         | Cons             |
|--------------------|------------------------------------|--------------|------------------|
| SMOTE (Oversample) | Creates synthetic minority samples | No data loss | Can create noise |
| Undersample        | Removes majority class samples     | Fast, simple | Loses real data  |

| Strategy                | How it works                   | Pros                  | Cons                   |
|-------------------------|--------------------------------|-----------------------|------------------------|
| class_weight='balanced' | Penalises minority errors more | No data change needed | Model-specific support |
| Collect more data       | Get more real minority samples | Best solution         | Not always possible    |

Amol Jagtap | amoljagtap3001@gmail.com

## 10. End-to-End Feature Engineering Workflow

In a real ML project, you never apply a single FE technique. You combine all the techniques learned in this session into a systematic pipeline. Below is the standard workflow used by data scientists.

| Step | Action                          | Technique                                    |
|------|---------------------------------|----------------------------------------------|
| 1    | Load and inspect data           | df.info(), df.describe(), df.isnull().sum()  |
| 2    | Handle missing values           | fillna(median/mean/mode), dropna()           |
| 3    | Detect and handle outliers      | IQR or Z-Score, clip or remove               |
| 4    | Encode categorical columns      | pd.get_dummies(), OrdinalEncoder             |
| 5    | Construct new features          | Date parts, ratios, interactions, binning    |
| 6    | Transform skewed distributions  | np.log1p(), PowerTransformer                 |
| 7    | Scale numeric features          | StandardScaler, MinMaxScaler (next video!)   |
| 8    | Select top features             | VarianceThreshold, RFE, feature_importances_ |
| 9    | Handle class imbalance (if any) | SMOTE, class_weight='balanced'               |
| 10   | Train the model                 | model.fit(X_train, y_train)                  |
| 11   | Evaluate                        | accuracy, F1, ROC-AUC                        |

### Putting it all together with sklearn Pipeline

```
from sklearn.pipeline import Pipeline from sklearn.preprocessing import StandardScaler, PowerTransformer from sklearn.feature_selection import SelectKBest, f_classif from sklearn.ensemble import RandomForestClassifier pipe = Pipeline([ ('power', PowerTransformer()), # fix skewness ('scaler', StandardScaler()), # scale features ('select', SelectKBest(f_classif, k=10)), # top 10 features ('model', RandomForestClassifier(n_estimators=100, class_weight='balanced')) ]) pipe.fit(X_train, y_train) print('Test Accuracy:', pipe.score(X_test, y_test)) # Save the whole pipeline (including all FE steps) import joblib joblib.dump(pipe, 'fe_pipeline.pkl')
```

■ Always fit preprocessing steps on training data ONLY, then transform both train and test. Fitting on test data causes data leakage — artificially inflated evaluation scores.

## 11. Quick Reference Cheat Sheet

| Task                         | Code                                                                       |
|------------------------------|----------------------------------------------------------------------------|
| Check skewness               | <code>df['col'].skew() # &gt; 1 or &lt; -1 = highly skewed</code>          |
| Log transform                | <code>np.log1p(df['col']) # <math>\log(1 + x)</math></code>                |
| Sqrt transform               | <code>np.sqrt(df['col'])</code>                                            |
| Box-Cox (auto power)         | <code>PowerTransformer(method='box-cox').fit_transform(X)</code>           |
| Yeo-Johnson                  | <code>PowerTransformer(method='yeo-johnson').fit_transform(X)</code>       |
| Extract year from date       | <code>df['Year'] = df['Date'].dt.year</code>                               |
| Extract weekday              | <code>df['Weekday'] = df['Date'].dt.weekday</code>                         |
| Is weekend flag              | <code>df['Is_Weekend'] = df['Weekday'].isin([5,6]).astype(int)</code>      |
| Age from DOB                 | <code>(pd.Timestamp.now() - df['DOB']).dt.days // 365</code>               |
| BMI                          | <code>df['BMI'] = df['Weight'] / df['Height']**2</code>                    |
| Bin continuous to groups     | <code>pd.cut(df['Age'], bins=[0,18,35,60], labels=['Y','M','S'])</code>    |
| Interaction feature          | <code>df['A_x_B'] = df['A'] * df['B']</code>                               |
| Variance threshold           | <code>VarianceThreshold(0.01).fit_transform(X)</code>                      |
| Correlation heatmap          | <code>sns.heatmap(df.corr(), annot=True, cmap='coolwarm')</code>           |
| SelectKBest (classification) | <code>SelectKBest(f_classif, k=10).fit_transform(X, y)</code>              |
| RFE                          | <code>RFE(LogisticRegression(), n_features_to_select=10)</code>            |
| Random Forest importance     | <code>rf.feature_importances_ (after fit)</code>                           |
| Lasso selection              | <code>Lasso(alpha=0.01).fit(X, y) -&gt; coef_ != 0</code>                  |
| PCA (2 components)           | <code>PCA(n_components=2).fit_transform(X_scaled)</code>                   |
| PCA (95% variance)           | <code>PCA(n_components=0.95).fit_transform(X_scaled)</code>                |
| LDA                          | <code>LinearDiscriminantAnalysis(n_components=2).fit_transform(X,y)</code> |
| IQR bounds                   | <code>Q1-1.5*IQR and Q3+1.5*IQR</code>                                     |
| Z-Score outliers             | <code>np.abs(stats.zscore(df['col'])) &gt; 3</code>                        |
| Clip outliers                | <code>df['col'].clip(lower, upper, inplace=True)</code>                    |
| Box plot check               | <code>sns.boxplot(data=df, y='col')</code>                                 |
| Check class distribution     | <code>df['target'].value_counts(normalize=True)</code>                     |
| SMOTE oversampling           | <code>SMOTE().fit_resample(X_train, y_train)</code>                        |



| Task                   | Code                                                                 |
|------------------------|----------------------------------------------------------------------|
| Random undersampling   | <code>RandomUnderSampler().fit_resample(X_train, y_train)</code>     |
| Balanced class weights | <code>LogisticRegression(class_weight='balanced')</code>             |
| Full FE pipeline       | <code>Pipeline([('scaler',...),('select',...),('model',...)])</code> |
| Save pipeline          | <code>joblib.dump(pipe, 'pipeline.pkl')</code>                       |

Amol Jagtap | amoljagtap3001@gmail.com

Amol Jagtap | amoljagtap3001@gmail.com